# SSH Key Authentication on AWS EC2

Experiment: Linux User Authentication Using SSH Keys

**Date: October 20, 2026**

## 1. Aim

To launch AWS EC2 Linux instances, generate an SSH key pair, create a Linux user, configure public-key authentication, and establish a secure SSH connection between instances.

## 2. Objectives

- To launch and configure EC2 Linux instances.
- To understand SSH key-pair authentication.
- To generate an Ed25519 public/private key pair.
- To create a Linux user and configure its SSH directory.
- To configure the `authorized_keys` file.
- To understand Linux file ownership and permissions.
- To transfer a private key to an intermediate EC2 instance and use it to connect to another instance.

## 3. Theory

SSH (Secure Shell) is a protocol used to securely connect to remote computers over a network. SSH encrypts communication between the client and the server.

SSH key authentication uses a cryptographic key pair:

- **Private key:** Stored on the client machine and used to prove possession of the secret key. It must be kept confidential.
- **Public key:** Installed on the server and authorized for a particular Linux account.
- **Authorized keys file:** The `authorized_keys` file contains public keys accepted for the corresponding Linux user.

The SSH client specifies the Linux username and the private key when initiating a connection. The SSH server verifies whether the matching public key is authorized for that account.

The following Linux commands are used in this experiment:

- `ssh-keygen`: Generates an SSH key pair.

- `ssh`: Establishes an SSH connection.
- `scp`: Transfers files securely over SSH.
- `chmod`: Changes file and directory permissions.
- `chown`: Changes file ownership.
- `useradd`: Creates a Linux user.
- `vi`: Edits text files in Linux.

## 4. Network Topology

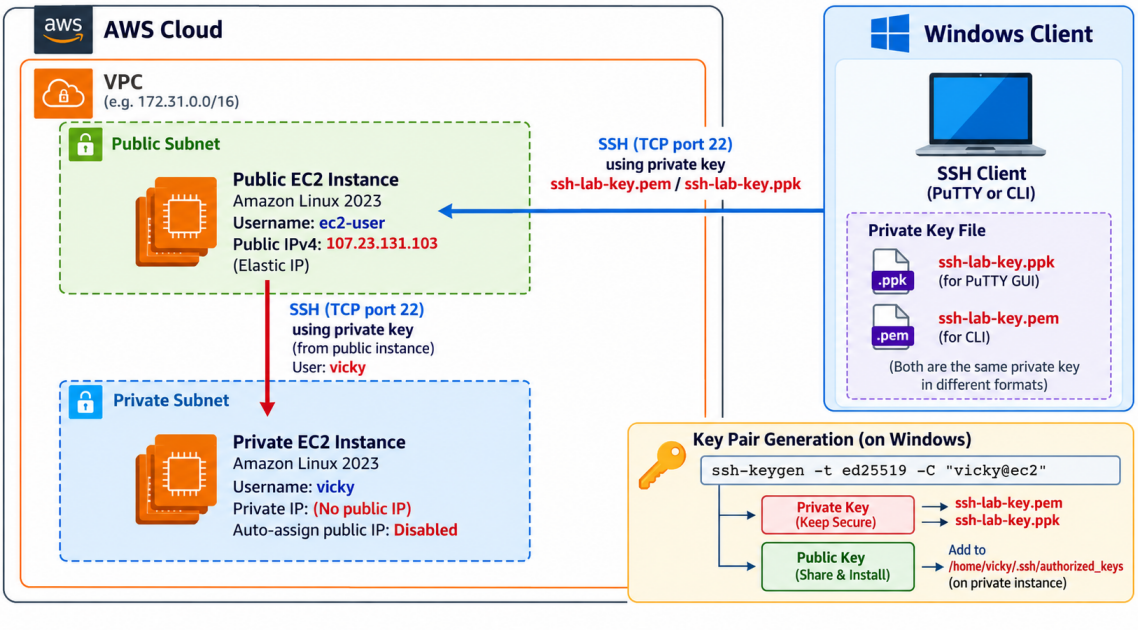



Figure 1: Network topology for SSH access from the public instance to the private EC2 instance.

## 5. Procedure

### 5.1 Step 1: Launch the EC2 Instance

The AWS EC2 Launch Instance wizard was opened in the AWS Management Console. The instance was named `PrivateServer`.

The Amazon Linux 2023 Amazon Machine Image (AMI) was selected, and the instance type was configured as `t3.micro`.

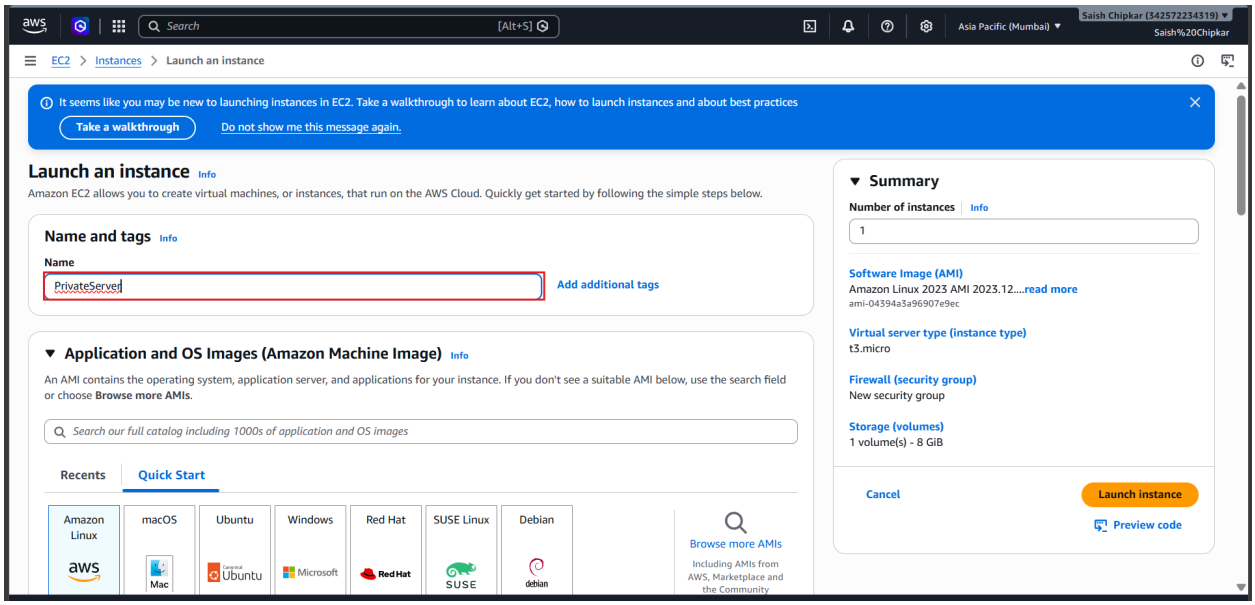



Figure 2: Naming the EC2 instance as PrivateServer.

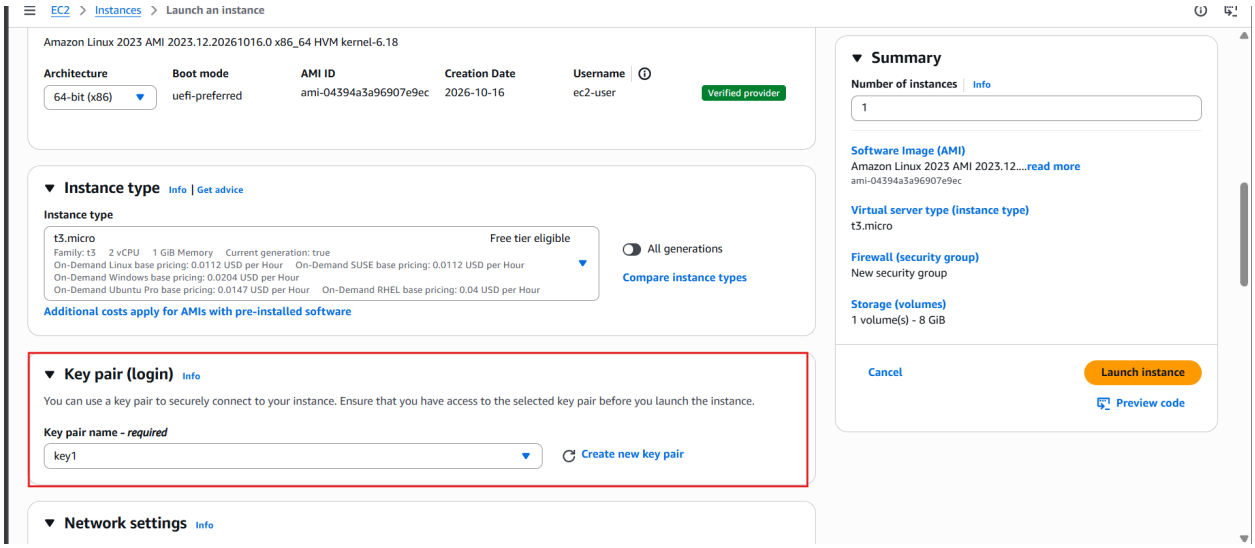



Figure 3: Selecting the Amazon Linux AMI, instance type, and EC2 key pair.

## 5.2 Step 2: Configure Network Settings

The default VPC and subnet settings were displayed during instance configuration. Auto-assign public IPv4 was disabled for the private instance.

This configuration allows the instance to use its private IPv4 address without automatically assigning a public IPv4 address.

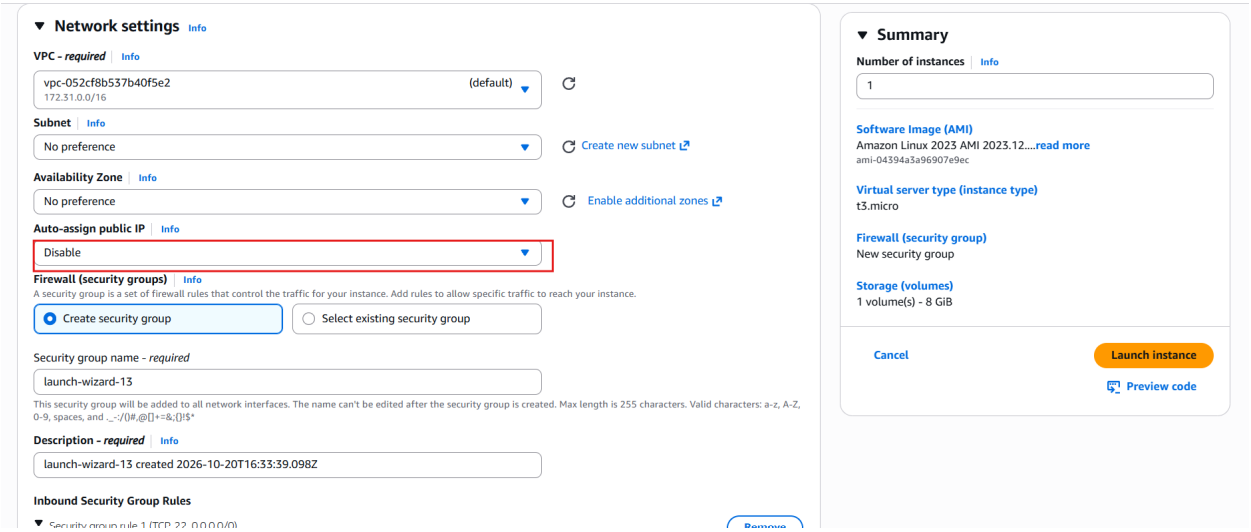



Figure 4: Configuring the VPC, subnet, and public IPv4 address assignment.

### 5.3 Step 3: Configure the Security Group

A security group was created for the instance. An inbound SSH rule was configured using TCP port 22.

The source was set to `0.0.0.0/0`, allowing SSH traffic from any IPv4 address. This setting is suitable only for a controlled demonstration and should normally be restricted to trusted source IP addresses.

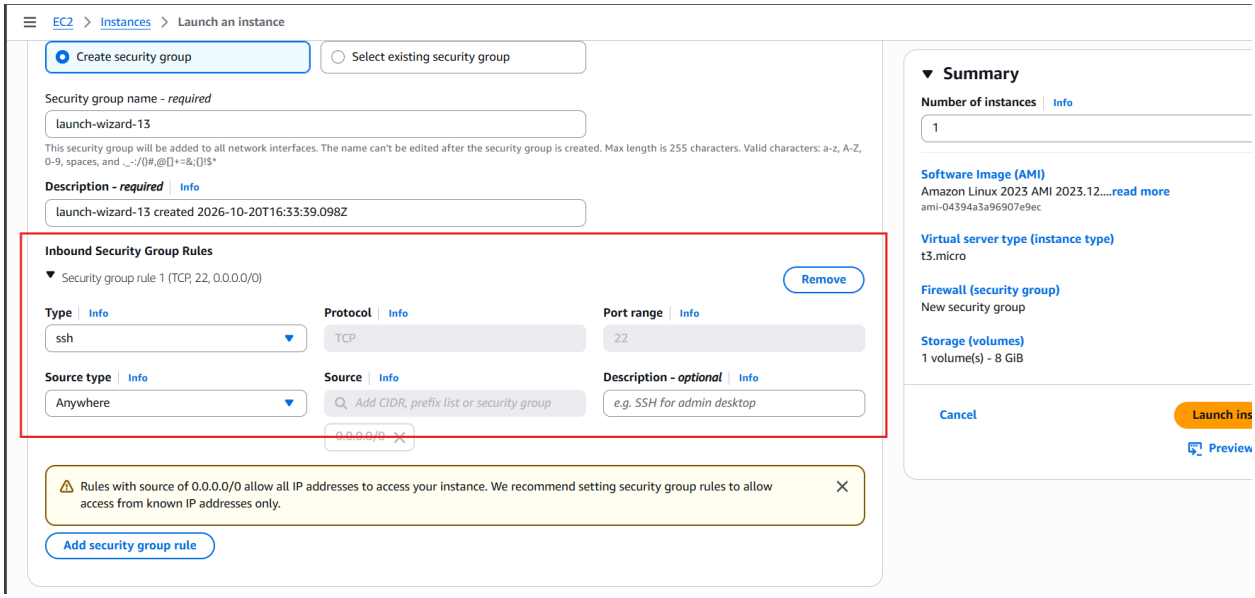



Figure 5: Configuring the inbound SSH security group rule.

### 5.4 Step 4: Verify the EC2 Instances

After launching the instances, the EC2 Instances page was opened to verify their states and network addresses.

The selected `PrivateServer` was running and had a private IPv4 address. No public IPv4 address was assigned to this instance.

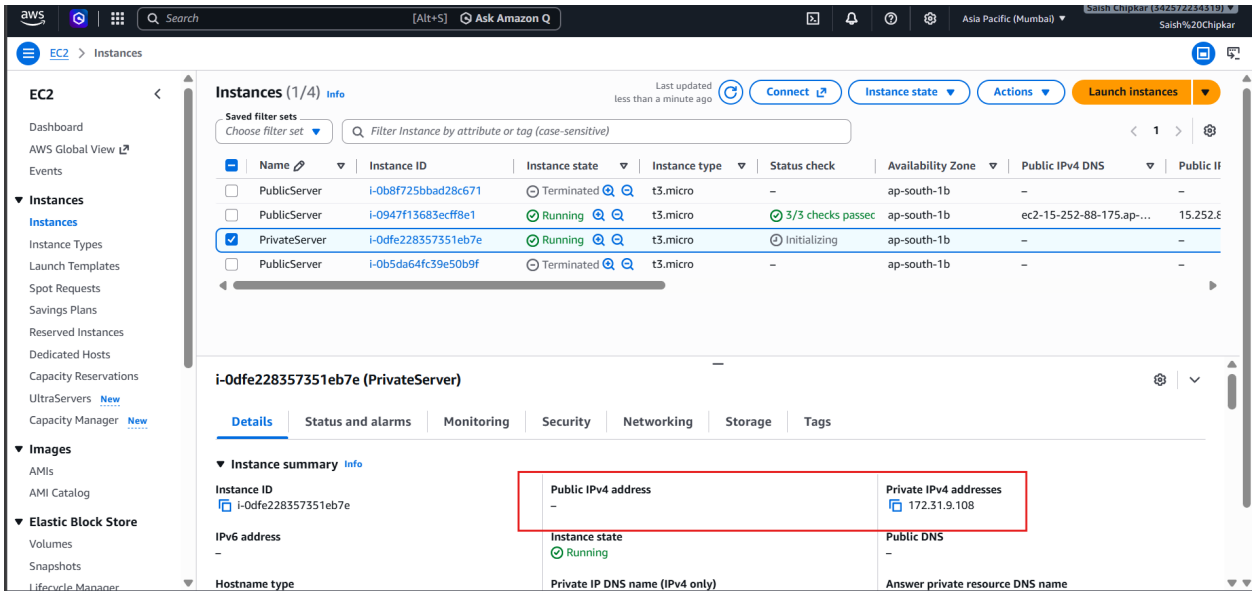



Figure 6: Verifying the running instances and the private IPv4 address.

### 5.5 Step 5: Transfer the Existing Private Key to the Public Instance

Windows PowerShell was used to transfer the existing EC2 private key file to the public instance using SCP.

The following command illustrates the transfer:

```
scp -i "<path-to-existing-key.pem>" ^
"<path-to-key-file>" saish@<Public-Instance-IP>:/home/saish/
```

The source file and destination were specified in the SCP command. The transfer completed successfully.

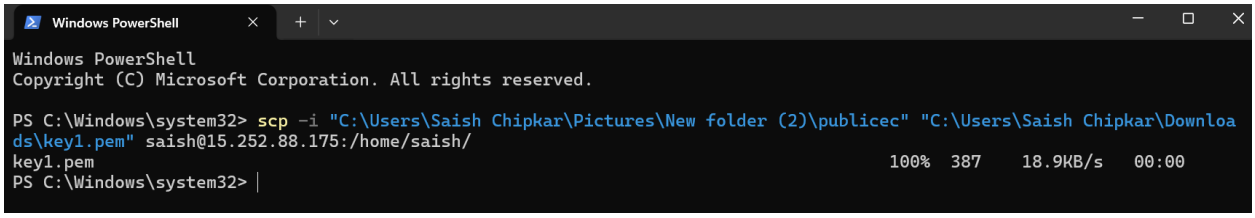



Figure 7: Transferring the existing EC2 private key using SCP.

### 5.6 Step 6: Connect to the Public Instance

SSH was used to connect to the public EC2 instance from Windows PowerShell.

```
ssh -i "<path-to-existing-key>" saish@<Public-Instance-IP>
```

After successful authentication, the Linux shell of the public instance was available for executing commands.

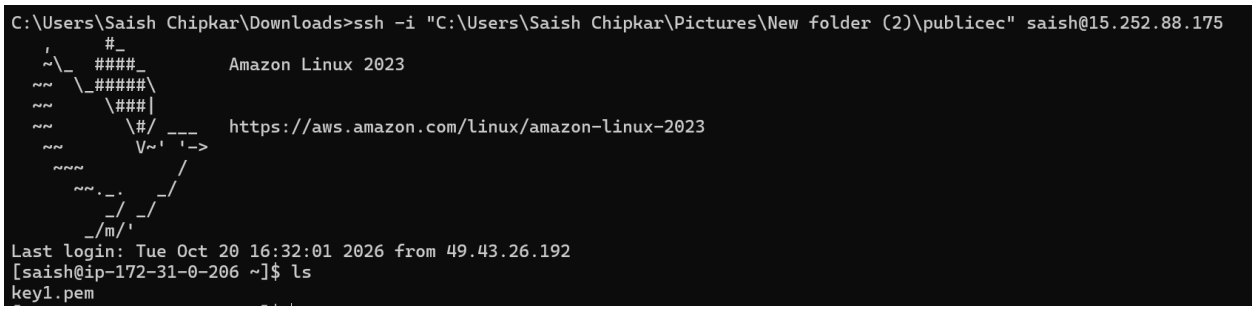



Figure 8: Connecting to the public EC2 instance through SSH.

### 5.7 Step 7: Check and Secure the Private Key File

The `ls -l` command was used to inspect the permissions of the transferred private key file.

Initially, the file had permissions that were too permissive for SSH private-key authentication. The following command was used to restrict access:

```
chmod 400 key1.pem
```

Permission `400` allows only the file owner to read the private key.

The file permissions were verified again using `ls -l`.

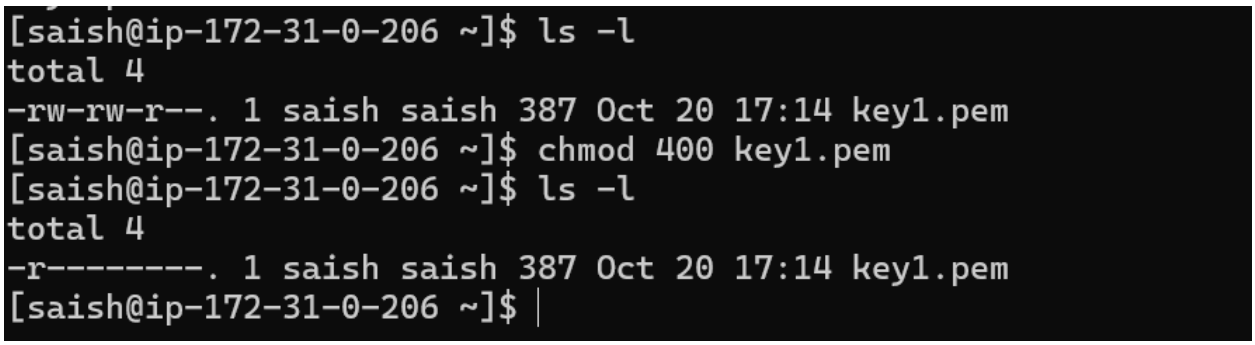



Figure 9: Checking and restricting the private key permissions.

### 5.8 Step 8: Connect to the Private Instance

From the public instance, SSH was used to connect to the private instance using its private IPv4 address.

```
ssh -i key1.pem ec2-user@<Private-IPv4-Address>
```

The public instance acted as an intermediate machine. The private instance was accessed through its private network address.

The command `whoami` was executed to verify the currently logged-in Linux user.

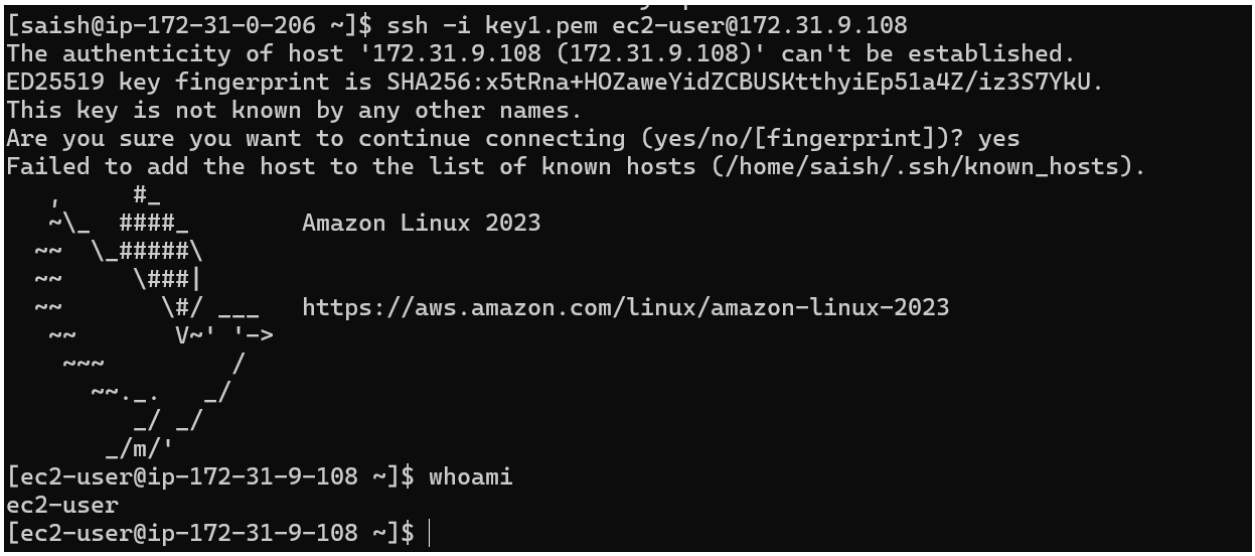



Figure 10: Connecting from the public instance to the private instance.

### 5.9 Step 9: Create a New Linux User

After connecting to the private instance, a new Linux user named `sunidhi` was created.

```
sudo adduser sunidhi
```

The `sudo` command was used to perform the operation with administrative privileges. The root shell was then entered to configure the new user's SSH directory.

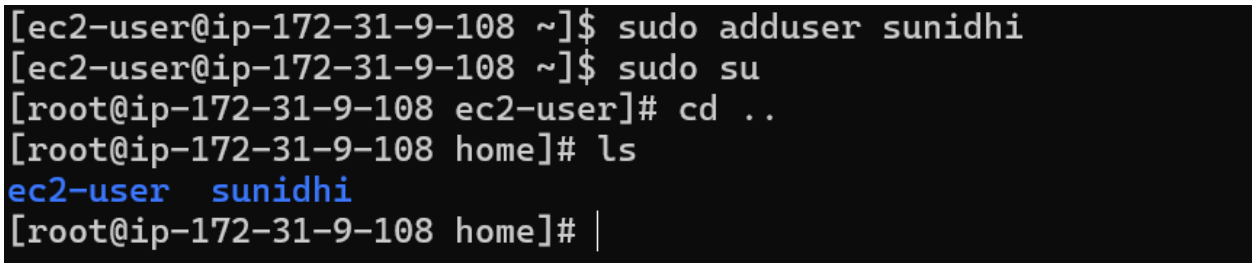



Figure 11: Creating the Linux user sunidhi.

### 5.10 Step 10: Create the SSH Directory

The home directory of the new user was accessed, and a hidden directory named `.ssh` was created.

```
cd /home/sunidhi
mkdir .ssh
ls -a
```

The `ls -a` command displays hidden files and directories, including those whose names begin with a dot.

```
[root@ip-172-31-9-108 home]# cd sunidhi
[root@ip-172-31-9-108 sunidhi]# mkdir .ssh
[root@ip-172-31-9-108 sunidhi]# ls
[root@ip-172-31-9-108 sunidhi]# ls -a
.  ..  .bash_logout  .bash_profile  .bashrc  .ssh
[root@ip-172-31-9-108 sunidhi]#
```

Figure 12: Creating and verifying the .ssh directory.

### 5.11 Step 11: Configure SSH Directory Ownership and Permissions

The ownership of the `.ssh` directory was assigned to the `sunidhi` user and group.

```
chown sunidhi:sunidhi .ssh
chmod 700 .ssh
```

The `chown` command changes ownership, while `chmod 700` grants read, write, and execute permissions to the owner and denies permissions to the group and others.

```
[root@ip-172-31-9-108 sunidhi]# chown sunidhi:sunidhi .ssh
[root@ip-172-31-9-108 sunidhi]# chmod 700 .ssh
```

Figure 13: Setting the ownership and permissions of the SSH directory.

### 5.12 Step 12: Generate a New SSH Key Pair on Windows

A new Ed25519 SSH key pair was generated on the Windows computer using the following command:

```
ssh-keygen -t ed25519 -C "sunidhi@ec2"
```

The private and public keys were saved at the specified location. The private key was retained on the Windows computer, while the public key was used to configure SSH access for the new Linux user.

```
PS C:\Windows\system32> ssh-keygen -t ed25519 -C "sunidhi@ec2"
Generating public/private ed25519 key pair.
Enter file in which to save the key (C:\Users\Saish Chipkar/.ssh/id_ed25519): C:\Users\Saish Chipkar\Pictures\sunidhikey
/sunikey
Enter passphrase (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in C:\Users\Saish Chipkar\Pictures\sunidhikey/sunikey
Your public key has been saved in C:\Users\Saish Chipkar\Pictures\sunidhikey/sunikey.pub
The key fingerprint is:
SHA256:uJ1DkA83yFVD3VM/SLA0bV4xcU5Z/nyiR8yqiDwCmws sunidhi@ec2
The key's randomart image is:
+--[ED25519 256]--+
|        .o+=oo =X|
|     . +  ..=o+*+|
|      * o  .o..o+|
|       * .   + .o|
|      . S     = +|
|  .    + .   + ..|
|E  +  . +   o .  |
| .o .... o . .   |
|  .. .o.. .      |
+----[SHA256]-----+
PS C:\Windows\system32>
```

Figure 14: Generating a new Ed25519 SSH key pair on Windows.

### 5.13 Step 13: Locate the Generated Public Key

The public key file generated by `ssh-keygen` was opened to obtain its contents.

The public key begins with the key type `ssh-ed25519` and ends with the comment `sunidhi@ec2`. Only the public key was used for server-side authorization.

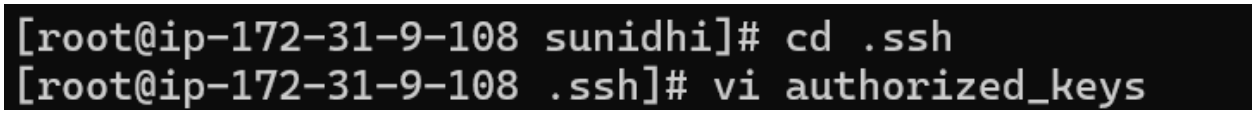



Figure 15: Viewing the generated public key.

### 5.14 Step 14: Configure the authorized_keys File

The public key was added to the authorized keys file of the `sunidhi` Linux account.

The file was opened using the `vi` editor:

```
cd /home/sunidhi/.ssh
vi authorized_keys
```

In the `vi` editor, the `i` key was pressed to enter Insert mode. The complete public key was pasted into the file. The Escape key was pressed, followed by `:wq` to save the file and exit the editor.

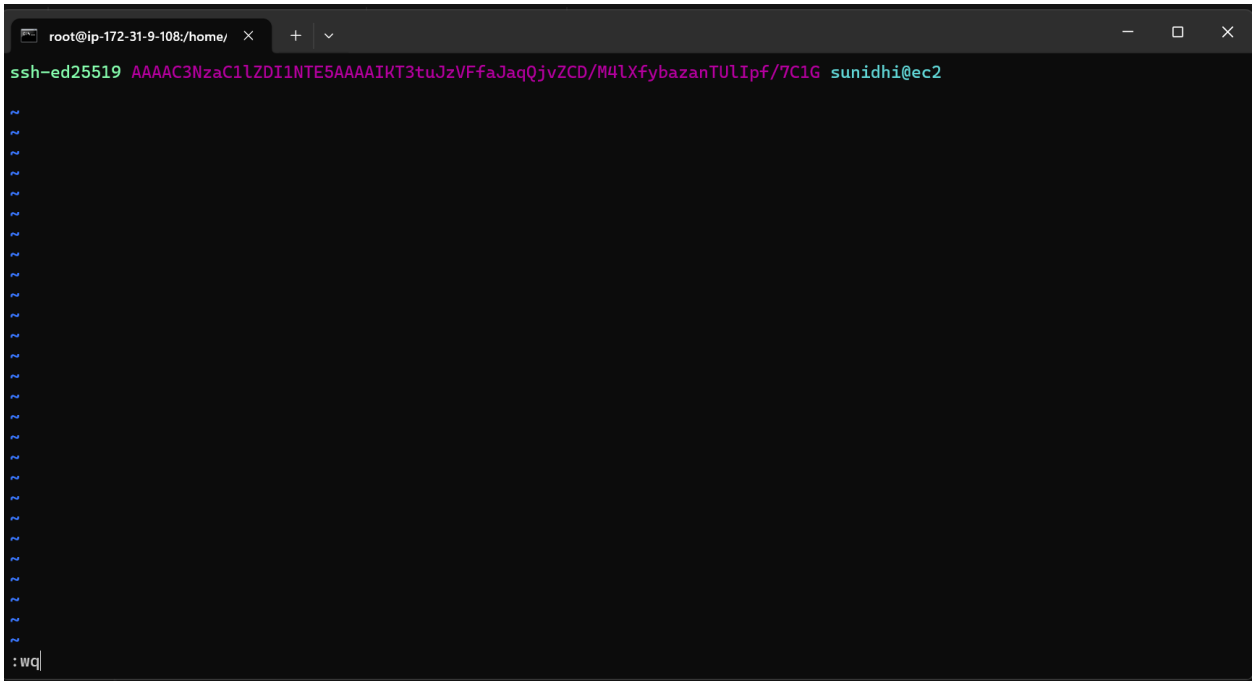



Figure 16: Editing the $\text{authorized}_{k}eysfileusingvi$.

### 5.15 Step 15: Set Ownership and Permissions for authorized_keys

The ownership of the authorized keys file was assigned to the `sunidhi` user and group.

```
chown sunidhi:sunidhi authorized_keys
chmod 600 authorized_keys
ls -l
```

Permission `600` grants read and write permissions to the owner and no permissions to the group or others.

These settings help prevent unauthorized users from modifying the keys authorized for the account.

```
[root@ip-172-31-9-108 .ssh]# chown sunidhi:sunidhi authorized_keys
[root@ip-172-31-9-108 .ssh]# chmod 600 authorized_keys
[root@ip-172-31-9-108 .ssh]# ls -l
total 4
-rw-------. 1 sunidhi sunidhi 400 Oct 20 17:31 authorized_keys
[root@ip-172-31-9-108 .ssh]#
```

Figure 17: Setting ownership and permissions for authorized$_keys$.

### 5.16 Step 16: Transfer the Generated Private Key to the Public Instance

The private key generated for the `sunidhi` account was transferred from Windows to the public EC2 instance using SCP.

The transfer command used the existing SSH access to the public instance and specified the destination directory.

```
scp -i "<path-to-existing-key.pem>" ^
"<path-to-generated-private-key>" saish@<Public-Instance-IP>:/home/saish/
```

The transfer was completed successfully.

```
PS C:\Windows\system32> scp -i "C:\Users\Saish Chipkar\Pictures\New folder (2)\publicec" "C:\Users\Saish Chipkar\Picture
s\sunidhikey\sunikey" saish@15.252.88.175:/home/saish/
sunikey                                                                    100%  399    24.4KB/s   00:00
```

Figure 18: Transferring the generated private key to the public instance.

### 5.17 Step 17: Verify the Transferred Private Key

After transferring the generated private key to the public EC2 instance, the `ls` command was used to verify that the files `key1.pem` and `sunikey` were present in the home directory of the `saish` user.

```
ls
ls -l
```

The file `sunikey` was the private key generated for the `sunidhi` Linux user. Its permissions were initially more permissive than required for SSH private-key authentication.

The following command was executed to restrict access:

```
chmod 400 sunikey
```

Permission `400` allows only the owner to read the private key. The `ls -l` command was executed again to verify the updated permissions.

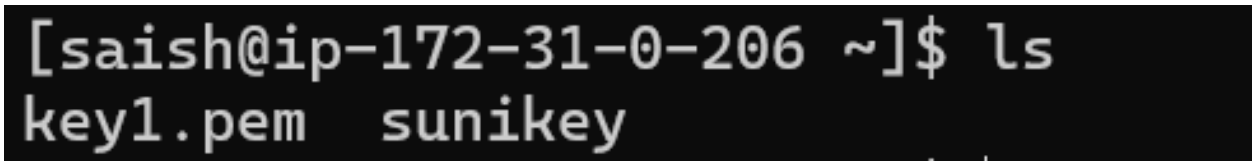



Figure 19: Verifying the transferred private key and its permissions.

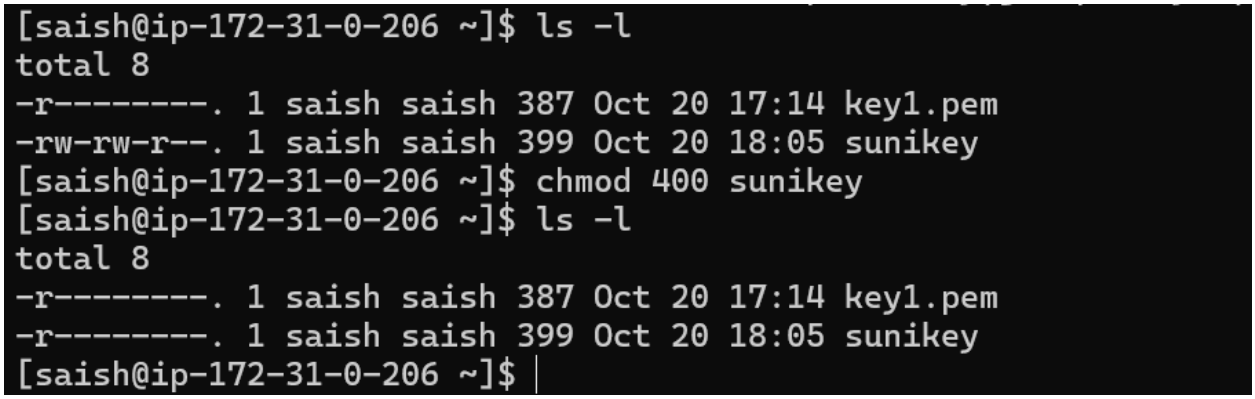



Figure 20: Restricting the private key permissions using chmod 400.

### 5.18 Step 18: Connect to the Private Instance as sunidhi

From the public EC2 instance, an SSH connection was initiated to the private instance using the newly generated private key `sunikey` and the Linux username `sunidhi`.

```
ssh -i sunikey sunidhi@172.31.9.108
```

The connection reached the private instance, and the shell prompt changed to indicate that the session was running as the `sunidhi` user.

A warning related to updating the `known_hosts` file was displayed because the SSH client could not update that file due to a permissions issue. Despite this warning, the session reached the private instance.

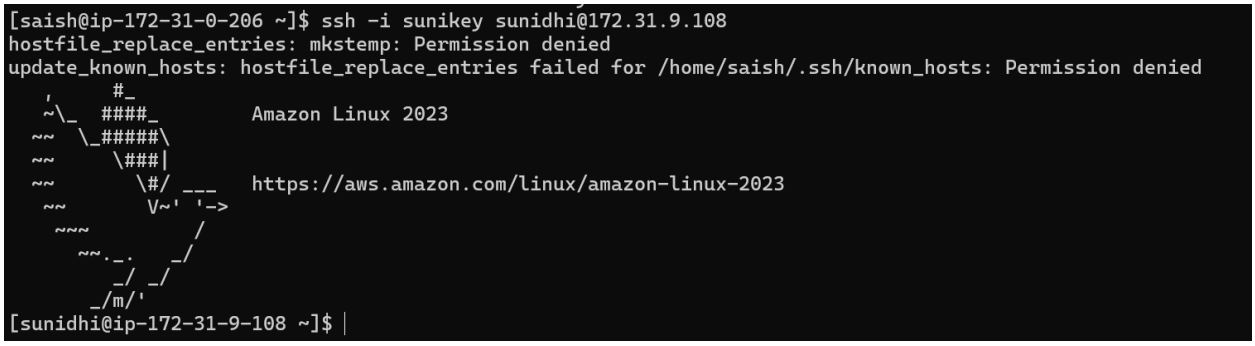



Figure 21: Connecting to the private instance using the sunidhi private key.

### 5.19 Step 19: Exit the SSH Sessions

After verifying access to the private instance, the SSH session was terminated using the `exit` command.

The root shell and the `ec2-user` session were exited sequentially. The connection to the private instance was closed, and the terminal returned to the shell prompt of the original public instance user, `saish`.

```
exit
exit
```

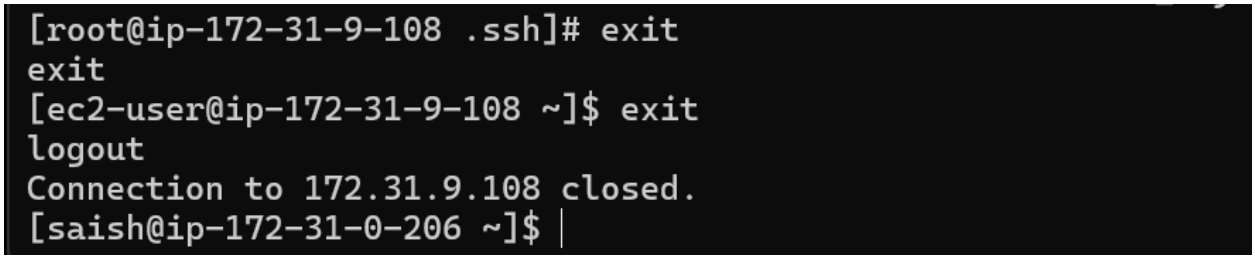



Figure 22: Exiting the SSH sessions and returning to the public instance.

## 6. Final Result

The SSH key authentication experiment was performed successfully. A new Linux user named `sunidhi` was created on the private EC2 instance, and the corresponding public key was added to the user's `authorized_keys` file.

The generated private key was transferred to the public EC2 instance and its permissions were restricted using `chmod 400`. An SSH connection was then initiated from the public instance to the private instance using the `sunidhi` account.

The SSH session reached the private instance, demonstrating the configuration of public-key authentication for the new Linux user.

## 7. Result

The EC2 instances were launched and configured. An SSH connection was established to the public instance, and the private instance was accessed through its private IPv4 address.

A new Linux user named `sunidhi` was created. An Ed25519 key pair was generated, and the corresponding public key was configured in the user's `authorized_keys` file with appropriate ownership and permissions.

The private key transfer to the public instance was completed as shown in the final screenshot.